

Session 3 · Deck 3.B1

Your Toolkit: Python, Environments, and Version Control

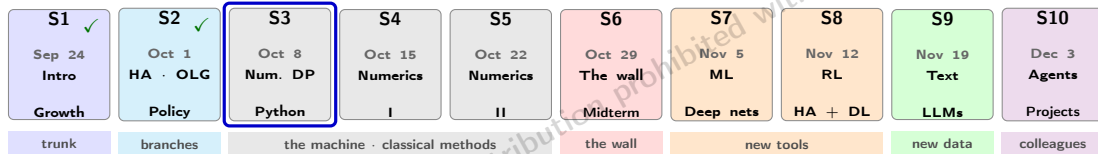
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.B1 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

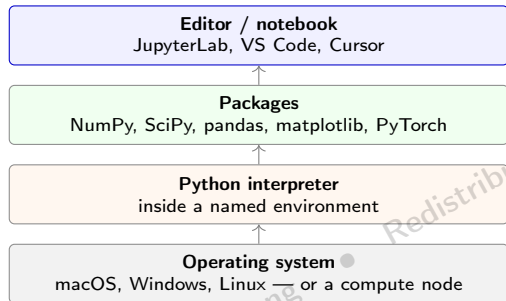
Block B: 3.B1 your toolkit · 3.B2 from loops to arrays · 3.B3 PyTorch, and coding with AI · 3.B4 the data workflow.

Four Error Messages You Are Going to Meet

What you will see	What it means	Fixed by
<code>ModuleNotFoundError: no module named 'numpy'</code>	the package is not installed <i>in the interpreter you are using</i>	environments
<code>command not found: python</code>	the shell cannot find an interpreter at all	installation, PATH
<code>AttributeError</code> on a function that exists in the docs	you are running a different version than the docs describe	environments
"It worked yesterday."	you changed something and cannot say what	version control

Not one of these is a Python problem. They are all **environment** problems, and they will cost you more hours this quarter than any algorithm in Block A. This deck is the twenty minutes that prevents them.

How the Pieces Fit Together



Adapted from QM Lec. 3, "How the Pieces Fit Together".

The distinction that matters

The editor is *not* the thing that runs your code. It sends text to an interpreter, and which interpreter it picks is a setting — usually the one you forgot to check.

Every “it works for you but not for me” in this course is a disagreement about one of the middle two layers.

Where You Will Actually Type

Tool	Best for
JupyterLab	exploring, plotting, homework you hand in
VS Code / Cursor	multi-file projects, debugging, AI assistance inline
PyCharm	large codebases, heavy refactoring

For this course: notebooks for the labs and homework, an editor once your project outgrows one file — which will happen in Session 5.

Adapted from QM-Lec. 3, “Jupyter, VS Code/Cursor, and PyCharm”.

A notebook is not a program

Cells can be run out of order, so a notebook that looks correct may depend on a variable you have since deleted.

Before you submit anything: **Restart Kernel and Run All**. If it fails, it was never working.

The same discipline is why the reference numbers in Block A were produced by a script submitted to a compute node, not by a notebook someone had been clicking around in.

Language, Library, Model — Three Different Things

- **Python** is the language: syntax, control flow, functions, classes. It is slow, and that is fine, because it is glue.
- **NumPy, SciPy, PyTorch** are libraries: compiled code with a Python interface. The arithmetic happens in C or CUDA, not in your loop.
- **An AI assistant** is a model behind an interface. It writes Python; it does not run it, and it does not know whether the answer is right.

Why this matters for speed

Python being slow is only a problem when *Python* does the arithmetic. Push the loop into the library and the language stops mattering — which is the whole of 3.B2.

And why it matters for trust: the library is tested by thousands of users; the assistant's output has been tested by nobody.

Adapted from QM, Lec. 3, "Python, PyTorch, and AI: How They Relate".

Installing: Who Does What

Anaconda / Miniconda bundles three separate things people confuse: a Python interpreter, a package manager, and an environment manager. Miniconda is the same thing without the 3 GB of packages you will not use.

An editor is not a distribution. Installing VS Code does not install Python; it goes looking for one. On

Windows, tick “Add to PATH”. Almost every command not found traces back to that box.

Adapted from QM Lec. 3, “Anaconda, Python, and Editors” and “Installing Python”.

```
$ python --version  
Python 3.11.9
```

```
$ which python  
/opt/miniconda3/envs/econ282e/bin/python
```

Run both of these *before* asking anyone for help. They answer “which Python am I actually using?”, which is the first question every answer depends on.

Environments: One Habit, Most of the Breakage Gone

An environment is a private folder holding one interpreter and one set of package versions. Projects stop fighting over versions because they never share any.

```
conda create -n econ282e python=3.11
conda activate econ282e
pip install numpy scipy pandas \
    matplotlib jupyter torch tqdm
pip freeze > requirements.txt
```

Two rules

1. Packages are installed *into* an environment. Installing while the wrong one is active is the single most common cause of `ModuleNotFoundError`.
2. Prefer conda *or* pip within one environment; mixing them for the same package leads to states neither tool can repair.

`requirements.txt` is what makes your results reproducible by someone else — including the version of you who returns in March.

The `install` commands follow QM Lec. 3, “Managing Packages”; environment creation is new — the source covers installing packages but never creating an environment to put them in.

Git: So That “It Worked Yesterday” Is a Question You Can Answer

Git records *snapshots* of a folder with a message attached. That is enough to answer the three questions that otherwise cost an afternoon: what changed, when, and what did it look like before.

```
git init
git add vfi.py
git commit -m "VFI matches Brock-Mirman"
git log --oneline
git diff HEAD~1
```

Commit when a number changes

The useful habit is not daily commits — it is committing at the moment a result becomes reproducible, with the result in the message.

Never commit data you do not own, model checkpoints, or anything with a password in it. A `.gitignore` is written once per project and saves you from all three.

Homework in this course is submitted as a PDF, the code that produced it, and an `AI_LOG.md`. Git is how you make sure those three agree.

Written for this course; version control does not appear anywhere in QM Lec. 3.

When You Need a Bigger Machine

Nothing in Block A needed more than a laptop: the entire morning's computation ran in under a second. That stops being true in Sessions 7 and 8, when training a network replaces iterating an operator.

- **Google Colab** gives a free GPU in a browser. Good for a lab, bad for anything you need to reproduce — the machine is gone when you close the tab.
- **A cluster** gives you a scheduler. You write a job script, submit it, and collect the output; the login node is for editing, never for computing.

The portable habit

Put the computation in a **script** that takes its settings from the top of the file and writes its results to disk. Then the same file runs on a laptop, in Colab, or under a scheduler, unchanged.

That is exactly how this morning's numbers were produced: one script, one output file, and every figure drawn from it.

Written for this course; Colab is mentioned in QM Lec. 2's programming-environment frame but not covered.

The Four Atomic Types

```
year      = 2026          # int
inflation = 0.031         # float
country   = "Chile"       # str
recession = False         # bool

type(inflation)           # <class 'float'>
int("2026") + 1           # 2027
float("3.1") * 2          # 6.2
```

Python infers the type; you do not declare it. That is convenient and occasionally dangerous.

1/2 is 0.5 in Python 3 but 0 in Python 2 and in most of C. Integer division is written `//` — and is exactly how you index into a grid.

A float here is the float64 of 3.A1: about sixteen significant digits, and gaps that widen as the numbers grow.

Adapted from QM Lec. 3, "Atomic Data Types".

Lists, Slicing, and Text

```
g = [2.1, 2.9, -0.4, 1.8, 2.5]

g[0]          # 2.1    first
g[-1]         # 2.5    last
g[1:3]        # [2.9, -0.4]
g[:2], g[3:]
len(g), sum(g) / len(g)

g.append(3.0)      # modifies g
sorted(g)[:2]      # two worst years

name = "  gdp Growth "
name.strip().title() # 'Gdp Growth'
f"inflation was {0.031:.1%}"
```

Adapted from QM Lec. 3, "Lists: Ordered Collections" and "Working with Text".

Slices are half-open: `g[1:3]` gives elements 1 and 2. It reads badly and it is the right convention — `g[:i] + g[i:]` reconstructs the list for every `i`.

Indexing from **zero** and slicing half-open are the two conventions behind every off-by-one error you will write this quarter. They are also what make `k[i]`, `k[i+1]` in an interpolation formula line up.

The One That Bites: Reference or Copy?

```
a = [1, 2, 3]
b = a           # NOT a copy
b[0] = 99
a               # [99, 2, 3]

c = a.copy()    # a real copy
c[0] = 1
a               # [99, 2, 3]
```

b = a makes a second *name* for one object.

Assignment never copies.

Why this will bite you

The same rule governs NumPy arrays and PyTorch tensors — and slices of them are **views**, not copies. In a value-function loop, `V_new = V` followed by an update silently destroys the very thing you wanted to compare against.

That is why 3.A2's loop writes `V, n = Vn, n + 1` — binding a *new* array — and never writes into the old one.

Adapted from QM Lec. 3, "Warning: Reference vs Copy".

Dictionaries, Tuples, Sets

```
par = {"alpha": 0.36, "beta": 0.95,  
       "delta": 1.0}  
par["beta"]           # 0.95  
par["sigma"] = 0.007  # add a key  
for k, v in par.items():  
    print(k, v)  
  
bounds = (0.05, 0.60) # tuple: fixed  
# bounds[0] = 0.1     -> TypeError  
  
set([1, 1, 2, 3])     # {1, 2, 3}
```

Adapted from QM Lec. 3, "Dictionaries: Key-Value Data" and "Tuples and Sets".

- **Dictionaries** are how you carry a calibration around. One object, named fields, and it serialises straight to JSON — which is how a results file records the parameters that produced it.
- **Tuples** are immutable. Use them for things that must not change: grid bounds, an array shape, a pair of coordinates.
- **Sets** deduplicate and test membership fast.

Takeaways

- The errors that cost you time this quarter will be **environment** errors, not Python errors. One environment per project, created on the first day, removes most of them.
- `which python` and `python --version` answer the question every other answer depends on. Run them before asking for help.
- The editor does not run your code and a notebook is not a program. **Restart and Run All** before you submit anything.
- Git is how “it worked yesterday” becomes a question with an answer. Commit when a number changes, and put the number in the message.
- Assignment never copies. `b = a` gives one object two names — and the same rule governs NumPy arrays and PyTorch tensors, where slices are views.

Bridge: The Algorithm Is on the Board. Put It in an Array.

You now have a machine that runs Python and a project that can be reproduced. What you do not have is the one idea that separates code which finishes from code which does not.

This morning's Bellman update is a double loop: for every state, for every choice. Written that way in Python it is unusably slow. Written as a single array expression it is the same arithmetic, the same answer to the last bit — and it runs about **65 times faster**.

Next (3.B2)

Control flow, functions and classes — then NumPy, broadcasting, and the vectorised Bellman update.

We rebuild this morning's solver as a `GrowthModel` class, and measure what each change bought.